

A Study and Development of Database Design for CRIME RECORD MANAGEMENT SYSTEM

CONTENTS

S.No	Topic
1	Abstarct
2	Introduction
	2.1 Objective
	2.2 Scope
	2.3 Technology used
	2.4 Scenario Description
3	Entities
	3.1 Types Of Entities
	3.2 List of entities For CRMS
4	Attributes
	4.1 Types of attributes
	4.2 Attributes Of CRMS
5	Relationships
	5.1 Types of Relationships
	5.2 Relationships of CRMS
6	ER diagram
	6.1 components of an ER diagram
	6.2 Entities summary
7	Data base design
	7.1 syntaxes for DDL commands
	7.2 create table for CRMS
8	Modifying data in the tables of CRMS
	8.1 syntaxes for DML commands
	8.2 Insert statements

	8.3 update statements
	8.4 delete statements
9	Data query language
10	Testing & project summary
	10.1 test cases
	10.2 summary report
	10.3 learning outcomes
	10.4 future enhancements
11	References

1.ABSTRACT

The Crime Record Management System (CRMS) is a database-driven application designed to digitize and streamline the management of crime-related information within a police department or law enforcement agency. The primary goal is to replace manual, paper-based record keeping with an efficient, structured, and easily queryable relational database.

The Crime Management System (CMS) is a relational database-driven application designed to digitize, organize, and streamline the processes involved in recording, tracking, and analyzing criminal activities. Law enforcement agencies worldwide deal with voluminous paper-based records that are difficult to search, update, and correlate. This project addresses that challenge by designing a normalized relational database backed by Structured Query Language (SQL).

This mini project leverages core concepts of Database Management Systems (DBMS) including Entity-Relationship (ER) modeling, normalization, Data Definition Language (DDL), Data Manipulation Language (DML), and Data Query Language (DQL). The system manages data about criminals, crimes, victims, FIRs, police officers, and police stations.

The system enables authorized police personnel to register FIRs, track criminal records, monitor ongoing investigations, and generate detailed reports. This project demonstrates how DBMS principles can be applied to a real-world, socially relevant scenario to enhance efficiency, data integrity, and accessibility of critical information.

2.INTRODUCTION

Law enforcement agencies handle enormous volumes of crime-related data every day. From FIR registration to trial proceedings, every step involves recording and retrieving critical information. Traditional paper-based systems are prone to data loss, redundancy, delays, and human error. A well-designed relational database resolves all these issues effectively.

2.1 OBJECTIVES

- Design and implement a relational database for managing crime records.
- Store and retrieve information about criminals, victims, FIRs, and officers.
- Demonstrate the use of DDL, DML, and DQL in a real-world context.
- Support efficient querying, filtering, and reporting of crime data.
- Ensure data integrity through constraints, primary keys, and foreign keys
- Translate the conceptual model into a normalized relational schema (3NF).

2.2 SCOPE

- FIR Registration and Tracking
- Criminal Profile Management
- Victim Information Management
- Police Officer and Station Management
- Crime Type and Case Status Tracking
- Investigation and Court Proceeding Records

2.3 TECHNOLOGY USED

Database	MySQL / Oracle
Query Language	SQL (DDL, DML, DQL)
ER Design Tool	draw.io / dbdiagram.io / MySQL Workbench
Documentation	MS Word

2.4 SCENARIO DESCRIPTION

Consider a City Police Department that manages crime records across multiple police stations. When a crime is reported, the following process takes place:

- A complainant visits a police station and files an FIR (First Information Report).
- The FIR is assigned a unique number and linked to a specific crime type (Theft, Murder, Fraud, Assault, etc.).
- One or more victims are recorded along with personal details.
- A police officer is assigned as the Investigating Officer (IO) for the case.
- If a suspect is identified, their profile is recorded including name, age, address, gender, and prior records.
- The case progresses through statuses: Open > Under Investigation > Charge Sheet > Trial > Closed.
- Court hearing details and judgment outcomes are stored for future reference. This scenario involves multiple entities with well-defined relationships, making it ideal for demonstrating DBMS concepts such as normalization, referential integrity, and complex SQL querying.

3. ENTITIES

- An **entity** is a clearly identifiable object such as a person, place, company, or concept
- In **Database Management Systems (DBMS)**, an **entity** is a real-world object, person, place, or concept that can be uniquely identified and about which data can be stored.

Each entity is described by a set of attributes that represent its properties

- Entities are represented as tables in a database, where each row corresponds to an instance of the entity and each column represents an attribute.

Entities play a fundamental role in database design, especially in the Entity-Relationship (ER) model

3.1 TYPES OF ENTITIES

- Strong Entity
- Weak Entity
- Associative Entity

STRONG ENTITY

- a **strong entity** is an entity that can be uniquely identified by its own attributes.
- It has a primary key that distinguishes each instance independently.
- A strong entity does not depend on any other entity for its existence.
- It is represented by a single rectangle in an ER diagram.

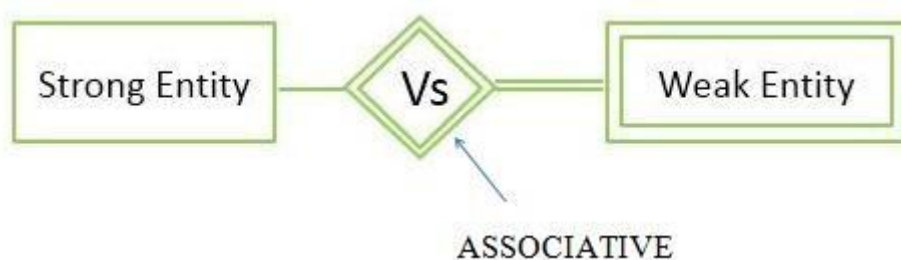
EX: Student, Employee, or Product.

WEAK ENTITY

- a **weak entity** is an entity that cannot be uniquely identified by its own attributes alone.
- It does not have a complete primary key and depends on a strong entity for identification.
- A weak entity is identified using a partial key along with the primary key of its related strong entity.
- It is existence-dependent on the strong entity. In an ER diagram, it is represented by a double rectangle.

ASSOCIATIVE ENTITY

- an **associative entity** is an entity that is created to represent a relationship between two or more entities.
- It is used when a many-to-many relationship needs to be converted into one-to-many relationships.
- An associative entity contains attributes that describe the relationship.
It has a primary key that is often a combination of the primary keys of the related entities.
It helps in organizing and managing complex relationships in a database.



3.2 LIST OF ENTITIES FOR CRMS

i. POLICE_STATION

ii. OFFICER iii.

CRIME iv.

CRIMINAL

v. VICTIM

vi. FIR

4. ATTRIBUTES

Attributes are the properties or characteristics of an entity. They describe the details of the entity and provide specific information about it. Each attribute corresponds to a column in a database table. Attributes have values for each entity instance (record).

EX: Name, Age, ID, and Address.

4.1 TYPES OF ATTRIBUTES

- **Simple Attribute** – Cannot be divided further (e.g., Age, ID).
- **Composite Attribute** – Can be divided into smaller sub-parts (e.g., FullName → FirstName + LastName).
- **Derived Attribute** – Value can be derived from other attributes (e.g., Age from DateOfBirth).
- **Single-Valued Attribute** – Holds only one value for an entity (e.g., Gender).
- **Multi-Valued Attribute** – Can hold multiple values for an entity (e.g., PhoneNumbers).
- **Key Attribute** – Uniquely identifies an entity (e.g., StudentID).

4.2 ATTRIBUTES OF CRMS

i. POLICE_STATION: station_id (PK), station_name, location, district, contact_no
ii. OFFICER: officer_id (PK), name, rank, badge_no, station_id (FK)
iii. CRIME: crime_id (PK), crime_type, crime_date, location, description, severity
iv. CRIMINAL: criminal_id (PK), name, dob, gender, address, prior_records, status

v. VICTIM:victim_id (PK), name, age, gender, contact_no, address **vi.**

FIR:fir_id (PK), fir_date, complainant_name, station_id (FK), crime_id (FK),officer_id (FK), case_status

5. RELATIONSHIPS

- A **relationship** refers to the association or connection between two or more entities (tables) in a database. Relationships show how data in one table is linked to data in another table using keys such as primary keys and foreign keys.
- A relationship in DBMS is the logical connection between tables that helps organize and retrieve related data efficiently.

5.1 TYPES OF RELATIONSHIPS

1. **One-to-One Relationship (1:1):** A relationship where one record in a table is related to only one record in another table.

Example: One person has one passport.

2. **One-to-Many Relationship (1:N):** A relationship where one record in a table can be related to multiple records in another table.

Example: One department has many employees.

3. **Many-to-One Relationship (N:1) :** A relationship where many records in one table are related to one record in another table.

Example: Many students belong to one college.

4. **Many-to-Many Relationship (M:N) :** A relationship where many records in one table are related to many records in another table.

Example: Students can enroll in many courses and each course can have many students.

5.2 RELATIONSHIPS OF CRMS

Relationship	Cardinality	Description
--------------	-------------	-------------

POLICE_STATION - OFFICER	1 : N	One station has many officers
OFFICER - FIR	1 : N	One officer handles many FIRs
CRIME - FIR	1 : 1	One crime is linked to one FIR
FIR - VICTIM	M : N	A FIR may have many victims
FIR - CRIMINAL	M : N	A FIR may involve many criminals
POLICE_STATION - FIR	1 : N	One station registers many FIRs

6. ER DIAGRAM (ENTITY RELATIONSHIPS)

An ER Diagram (Entity–Relationship Diagram) in Database Management System is a graphical representation of entities, attributes, and relationships in a database.

It is used to design and visualize the structure of a database before creating the tables.

Example :

In a crime management system, entities like Victim, FIR, Officer, and Police Station are connected through relationships such as files, handles, or belongs to in an ER diagram.

6.1 COMPONENTS OF AN ER DIAGRAM

- Entity
- Attribute
- Relationship
- Cardinality

6.2 ENTITIES SUMMARY

- **POLICE_STATION** (station_id PK, station_name, location, district, contact_no)
- **OFFICER** (officer_id PK, name, rank, badge_no, station_id FK)
- **FIR** (fir_id PK, fir_date, complainant_name, station_id FK, crime_id FK, officer_id FK, case_status)
- **CRIME** (crime_id PK, crime_type, crime_date, location, description, severity)

- **CRIMINAL** (criminal_id PK, name, dob, gender, address, prior_records, status)
- **VICTIM** (victim_id PK, name, age, gender, contact_no, address)

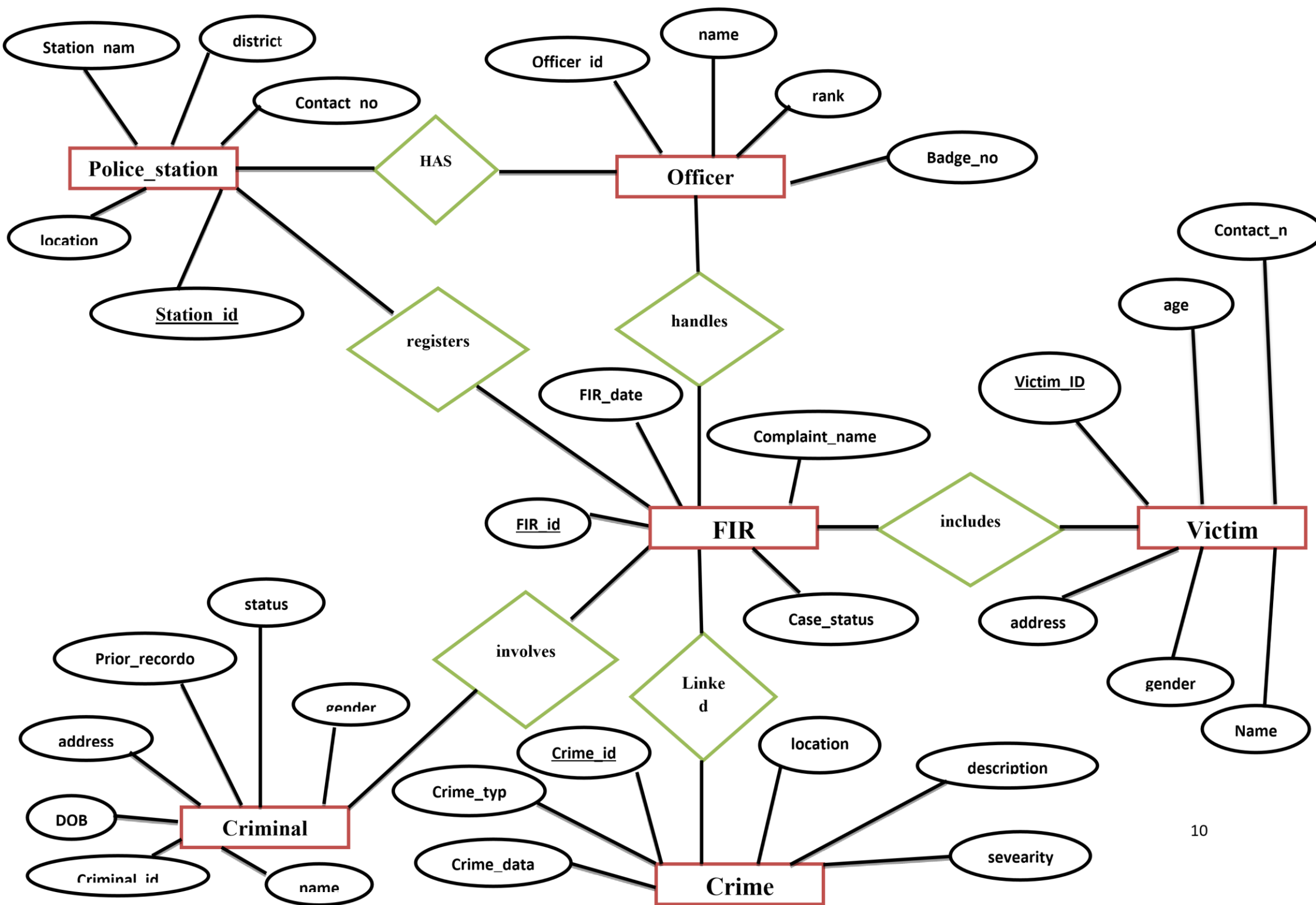
Junction Tables (M:N Resolution)

- **FIR_CRIMINAL** (fir_id FK, criminal_id FK) -- resolves M:N between FIR and CRIMINAL
 - **FIR_VICTIM** (fir_id FK, victim_id FK) -- resolves M:N between FIR and VICTIM
- From the relationships you gave, the **verbs (relationship words)** are:

- **POLICE_STATION – OFFICER → has**
- **OFFICER – FIR → handles**
- **CRIME – FIR → linked to**
- **FIR – VICTIM → includes**
- **FIR – CRIMINAL → involves**
- **POLICE_STATION – FIR → registers**

The relationships among these entities in **Database Management System** are:

1. **POLICE_STATION – OFFICER → One-to-Many (1 : N)**
One police station **has** many officers.
2. **OFFICER – FIR → One-to-Many (1 : N)** One officer can **handle** many FIRs.
3. **CRIME – FIR → One-to-One (1 : 1)** One crime is **linked** to one FIR.
4. **FIR – VICTIM → Many-to-Many (M : N)**
One FIR may **include** many victims and one victim may appear in multiple FIRs.
5. **FIR – CRIMINAL → Many-to-Many (M : N)**
One FIR may **involve** many criminals and one criminal may be involved in multiple FIRs.
6. **POLICE_STATION – FIR → One-to-Many (1 : N)**
One police station can **register** many FIR



7.DATABASE DESIGN

(DDL- Data definition language)

Database Design (DDL) refers to the use of **Data Definition Language** statements to **define and create the structure of a database**. It specifies tables, columns, data types, and constraints.

Common DDL commands:

- **CREATE** – Creates database objects such as tables.
- **ALTER** – Modifies the structure of an existing table.
- **DROP** – Deletes tables or other database objects.
- **TRUNCATE** – Removes all records from a table.

7.1 SYNTAXES FOR DDL COMMANDS

1. CREATE: Used to create a new table.

```
CREATE TABLE table_name (  
column1 datatype, column2  
datatype, column3 datatype  
);
```

2. ALTER : Used to modify an existing table.

```
ALTER TABLE table_name  
ADD column_namedatatype;
```

3. DROP : Used to delete a table completely from the database.

```
DROP TABLE table_name;
```

4. TRUNCATE : Used to remove all records from a table but keep the structure.

```
TRUNCATE TABLE table_name;
```

7.2 CREATE TABLE FOR CRMS

POLICE_STATION

```
CREATE TABLE POLICE_STATION (  
station_id VARCHAR2(6) PRIMARY KEY,  
station_name VARCHAR2(100) NOT NULL,
```

```
location VARCHAR2(150) NOT NULL, district
VARCHAR2(80) NOT NULL,
contact_no VARCHAR2(15) UNIQUE
);
```

OFFICER

```
CREATE TABLE OFFICER (
officer_id VARCHAR2(6) PRIMARY KEY,
name VARCHAR2(100) NOT NULL, rank
VARCHAR2(50) NOT NULL,
badge_no VARCHAR2(20) UNIQUE,
station_id VARCHAR2(6),
FOREIGN KEY (station_id) REFERENCES POLICE_STATION(station_id)
);
```

CRIME

```
CREATE TABLE CRIME (
crime_id VARCHAR2(6) PRIMARY KEY,
crime_type VARCHAR2(80) NOT NULL,
crime_date DATE NOT NULL, location
VARCHAR2(150) NOT NULL, description
CLOB, severity VARCHAR2(10) DEFAULT
'Medium',
CHECK (severity IN ('Low','Medium','High'))
);
```

CRIMINAL

```
CREATE TABLE CRIMINAL (
criminal_id VARCHAR2(6) PRIMARY KEY,
name VARCHAR2(100) NOT NULL, dob DATE,
```

```

gender CHAR(1) CHECK (gender IN ('M','F','O')),
address VARCHAR2(200), prior_records
NUMBER DEFAULT 0, status VARCHAR2(30)
DEFAULT 'Arrested'
);

```

```

VICTIM CREATE TABLE VICTIM (
victim_id VARCHAR2(6) PRIMARY KEY, name
VARCHAR2(100) NOT NULL, age NUMBER
CHECK (age > 0), gender CHAR(1) CHECK
(gender IN ('M','F','O')),
contact_no VARCHAR2(15), address
VARCHAR2(200)
);

```

```

FIR CREATE TABLE FIR ( fir_id VARCHAR2(6)
PRIMARY KEY, fir_date DATE NOT NULL,
complainant_name VARCHAR2(100) NOT NULL,
station_id VARCHAR2(6),
crime_id VARCHAR2(6),
officer_id VARCHAR2(6),
case_status VARCHAR2(30) DEFAULT 'Open',
CHECK (case_status IN ('Open','UnderInvestigation','ChargeSheeted','Trial','Closed')),
FOREIGN KEY (station_id) REFERENCES POLICE_STATION(station_id),
FOREIGN KEY (crime_id) REFERENCES CRIME(crime_id),
FOREIGN KEY (officer_id) REFERENCES OFFICER(officer_id)

```

);

FIR_CRIMINAL (Junction Table) CREATE

```
TABLE FIR_CRIMINAL (  
    fir_id VARCHAR2(6),  
    criminal_id VARCHAR2(6),  
    PRIMARY KEY (fir_id, criminal_id),  
    FOREIGN KEY (fir_id) REFERENCES FIR(fir_id),  
    FOREIGN KEY (criminal_id) REFERENCES CRIMINAL(criminal_id)  
);
```

FIR_VICTIM (Junction Table) CREATE

```
TABLE FIR_VICTIM (  
    fir_id VARCHAR2(6),  
    victim_id VARCHAR2(6),  
    PRIMARY KEY (fir_id, victim_id),  
    FOREIGN KEY (fir_id) REFERENCES FIR(fir_id),  
    FOREIGN KEY (victim_id) REFERENCES VICTIM(victim_id)  
);
```

8.MODIFYING DATA IN THE TABLES OF CRMS

In Database Management System, Data Manipulation Language (DML) refers to the set of SQL commands used to manipulate the data stored in database tables.

It allows users to **insert, update, delete, and retrieve data** from the database.

Common DML Commands

- **INSERT** – Adds new records into a table

- **UPDATE** – Modifies existing records
- **DELETE** – Removes records from a table
- **SELECT** – Retrieves data from the database

8.1 SYNTAXES FOR DML COMMANDS

1. INSERT :Used to add new records into a table.

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

2. UPDATE :Used to modify existing records in a table.

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

3. DELETE :Used to remove records from a table.

```
DELETE FROM table_name
WHERE condition;
```

4. SELECT :Used to retrieve data from a table.

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

8.2 INSERT STATEMENTS

POLICE STATIONS

```
INSERT INTO POLICE_STATION VALUES ('P1','Tadepalligudem Town PS','Main Road,
Tadepalligudem','West Godavari','08818-224455');
```

```
INSERT INTO POLICE_STATION VALUES ('P2','Bhimavaram PS','NH-16,
Bhimavaram','West Godavari','08816-223344');
```

```
INSERT INTO POLICE_STATION VALUES ('P3','Eluru PS','Eluru Central, Eluru','Eluru
District','08812-233445'); select * from police_station;
```

STATION_ID	STATION_NAME	LOCATION	DISTRICT	CONTACT_NO
P1	Tadepalligudem Town PS	Main Road, Tadepalligudem	West Godavari	08818-224455
P2	Bhimavaram PS	NH-16, Bhimavaram	West Godavari	08816-223344
P3	Eluru PS	Eluru Central, Eluru	Eluru District	08812-233445

OFFICERS

INSERT INTO OFFICER VALUES ('O1','Ravi Kumar','Sub Inspector','SI-1021','P1');
 INSERT INTO OFFICER VALUES ('O2','Lakshmi Devi','Inspector','INS-0456','P2');
 INSERT INTO OFFICER VALUES ('O3','Suresh Babu','Head Constable','HC-7893','P1');
 INSERT INTO OFFICER VALUES ('O4','Anand Rao','DSP','DSP-0089','P3'); **select ***
from officer;

OFFICER_ID	NAME	RANK	BADGE_NO	STATION_ID
O1	Ravi Kumar	Sub Inspector	SI-1021	P1
O2	Lakshmi Devi	Inspector	INS-0456	P2
O3	Suresh Babu	Head Constable	HC-7893	P1
O4	AnandRao	DSP	DSP-0089	P3

CRIMES

INSERT INTO CRIME VALUES ('C1','Theft', DATE '2025-01-10','Market Area, Tadepalligudem','Mobile and cash stolen','Medium');
 INSERT INTO CRIME VALUES ('C2','Murder', DATE '2025-02-14','Railway Station, Bhimavaram','Stabbing incident','High');
 INSERT INTO CRIME VALUES ('C3','Cybercrime', DATE '2025-03-05','Online','Bank fraud via phishing','High');
 INSERT INTO CRIME VALUES ('C4','Assault', DATE '2025-04-22','Bus Stand, Eluru','Physical altercation','Low'); **select**
*** from crime;**

CRI ME_I D	CRIME_TYPE	CRIME_DATE	LOCATION	DESCRIPTION	SEVERITY

C1	Theft	10-JAN-25	Market Area, Tadepalligudem	Mobile and cash stol en	Medium
C2	Murder	14-FEB-25	Railway Station, Bhimavaram	Stabbing incident	High
C3	Cybercrime	05-MAR-25	Online	Bank fraud via phish ing	High
C4	Assault	22-APR-25	Bus Stand, Eluru	Physical altercation	Low

CRIMINALS

INSERT INTO CRIMINAL VALUES ('CR1','Ramesh Goud', DATE '1990-06-15','M','Old Town, Tadepalligudem',2,'Arrested');

INSERT INTO CRIMINAL VALUES ('CR2','Sunil Das', DATE '1985-11-20','M','Krishna Nagar, Bhimavaram',1,'Absconding');

INSERT INTO CRIMINAL VALUES ('CR3','Praveen Raj', DATE '1995-03-10','M','Eluru Road, Eluru',0,'Arrested');

select * from criminal;

CRIMINAL_ID	NAME	DOB	GEN	ADDRESS	PRIOR_RECORDS	STATUS
CR1	Ramesh Goud	15JUN-90	M	Old Town, Tadepalligudem	2	Arrested
CR2	Sunil Das	20-NOV-85	M	Krishna Nagar, Bhimavaram	1	Absconding
CR3	Praveen Kumar	10-MAR-95	M	Eluru Road, Eluru	0	Arrested

VICTIMS

INSERT INTO VICTIM VALUES ('V1','Sita Devi',42,'F','9876543210','Tadepalligudem');

INSERT INTO VICTIM VALUES ('V2','Mohan Rao',30,'M','9988776655','Bhimavaram');

INSERT INTO VICTIM VALUES ('V3','Priya Krishnan',25,'F','9123456789','Eluru'); **select * from victim;**

VICTIM_ID	NAME	AGE	GEN	CONTACT_NO	ADDRESS
V1	Sita Devi	42	F	9876543210	Tadepalligudem
V2	Mohan Rao	30	M	9988776655	Bhimavaram
V3	Priya Krishnan	25	F	9123456789	Eluru

FIR

INSERT INTO FIR VALUES ('F1', DATE '2025-01-11','Sita Devi','P1','C1','O1','Under Investigation');

INSERT INTO FIR VALUES ('F2', DATE '2025-02-15','Mohan Rao','P2','C2','O2','Charge Sheeted');

INSERT INTO FIR VALUES ('F3', DATE '2025-03-06','Priya Krishnan','P2','C3','O2','Open');

INSERT INTO FIR VALUES ('F4', DATE '2025-04-23','Raju Yadav','P3','C4','O4','Closed');

select * from fir;

FIR_ID	FIR_DATE	COMPLAINANT_NAME	STATION_ID	CRIME_ID	OFFICER_ID	CASE_STATUS
F1	11-JAN-25	Sita Devi	P1	C1	O1	Trial
F2	15-FEB-25	Mohan Rao	P2	C2	O2	Charge Sheeted
F3	06-MAR-25	Priya Krishnan	P2	C3	O2	Open
F4	23-APR-25	RajuYadav	P3	C4	O4	Closed

Junction Tables

FIR_CRIMINAL

INSERT INTO FIR_CRIMINAL VALUES ('F1','CR1');

INSERT INTO FIR_CRIMINAL VALUES ('F2','CR2');

INSERT INTO FIR_CRIMINAL VALUES ('F3','CR3');

INSERT INTO FIR_CRIMINAL VALUES ('F4','CR1');

select * from fir_criminal;

FIR_ID	CRIMINAL_ID
F1	CR1
F2	CR2
F3	CR3
F4	CR1

FIR_VICTIM

INSERT INTO FIR_VICTIM VALUES ('F1','V1');

INSERT INTO FIR_VICTIM VALUES ('F2','V2');

INSERT INTO FIR_VICTIM VALUES ('F3','V3');

INSERT INTO FIR_VICTIM VALUES ('F4','V1');

select * from fir_victim;

FIR_ID	VICTIM_ID
F1	V1

F2	V2
F3	V3
F4	V1

8.3 UPDATE STATEMENTS

-- Update case status of FIR No. F1

UPDATE FIR

SET case_status = 'Trial'

WHERE fir_id = 'F1';

FIR_ID	FIR_DATE	COMPLAINANT_NAME	STATION_ID	CRIME_ID	OFFICER_ID	CASE_STATUS
F1	11-JAN-25	Sita Devi	P1	C1	O1	Trial
F2	15-FEB-25	Mohan Rao	P2	C2	O2	Charge Sheeted
F3	06-MAR-25	Priya Krishnan	P2	C3	O2	Open
F4	23-APR-25	RajuYadav	P3	C4	O4	Closed

-- Update criminal status after arrest

UPDATE CRIMINAL

SET status = 'Arrested'

WHERE criminal_id = 'CR2';

1 row updated.

CRIMINAL_ID	NAME	DOB	GEN	ADDRESS	PRIOR_RECORDS	STATUS
CR1	Ramesh Goud	15JUN-90	M	Old Town, Tadepalligudem	2	Arrested
CR2	Sunil Das	20-NOV-85	M	Krishna Nagar, Bhimavaram	1	Arrested
CR3	Praveen Kumar	10-MAR-95	M	Eluru Road, Eluru	0	Arrested

-- Promote officer rank

UPDATE OFFICER

SET rank = 'Inspector'

WHERE officer_id = 'O1';

1 row updated.

OFFICER_ID	NAME	RANK	BADGE_NO	STATION_ID
------------	------	------	----------	------------

O1	Ravi Kumar	Inspector	SI-1021	P1
O2	Lakshmi Devi	Inspector	INS-0456	P2
O3	Suresh Babu	Head Constable	HC-7893	P1
O4	AnandRao	DSP	DSP-0089	P3

8.4 DELETE STATEMENTS

-- Remove closed FIR (delete child records first)

DELETE FROM FIR_CRIMINAL WHERE fir_id = 'F4';

DELETE FROM FIR_VICTIM WHERE fir_id = 'F4';

DELETE FROM FIR WHERE fir_id = 'F4';

Output

1 rows deleted.

1 rows deleted.

1 rows deleted.

9. DATA QUERY LANGUAGE (DQL)

, Data Query Language (DQL) refers to the set of SQL commands used to retrieve or fetch data from a database. DQL allows users to query the database and view the stored information without modifying it.

SELECT – used to retrieve data from one or more tables.

Example: SELECT * FROM table_name;

Basic SELECT Queries

Q1. Display all FIRs

SELECT * FROM FIR;

FIR_ID	FIR_DATE	COMPLAINANT_NAME	STATION_ID	CRIME_ID	OFFICER_ID	CASE_STATUS
F1	11-JAN-25	Sita Devi	P1	C1	O1	Trial
F2	15-FEB-25	Mohan Rao	P2	C2	O2	Charge Sheeted
F3	06-MAR-25	Priya Krishnan	P2	C3	O2	Open

Q2. List all Open FIR cases

SELECT fir_id, fir_date, complainant_name, case_status
FROM FIR
WHERE case_status = 'Open';

FIR_ID	FIR_DATE	COMPLAINANT_NAME	CASE_STATUS
F3	06-MAR-25	Priya Krishnan	Open

Q3. Criminals who are Absconding

SELECT criminal_id, name, address
FROM CRIMINAL
WHERE status = 'Absconding';
no
rows selected

Q4. Total FIRs per police station

SELECT ps.station_name, COUNT(f.fir_id) AS total_firs
FROM POLICE_STATION ps
LEFT JOIN FIR f ON ps.station_id = f.station_id
GROUP BY ps.station_name;

STATION_NAME	TOTAL_FIRS
Eluru PS	0
Tadepalligudem Town PS	1
Bhimavaram PS	2

JOIN Queries

Q5. Full FIR details with Crime, Officer and Station

SELECT f.fir_id, f.fir_date, f.complainant_name,
c.crime_type, c.severity,

```

o.name AS officer_name, o.rank, ps.station_name,
f.case_status
FROM FIR f
JOIN CRIME c ON f.crime_id = c.crime_id
JOIN OFFICER o ON f.officer_id = o.officer_id
JOIN POLICE_STATION ps ON f.station_id = ps.station_id;

```

FIR_ID	FIR_DATE	COMPLAINANT_NAME	CRIME_TYPE	SEVERITY	OFFICER_NAME	RANK	STATION_NAME	CASE_STATUS
F1	11JAN-25	Sita Devi	Theft	Medium	Ravi Kumar	Inspector	Tadepalligudem Town PS	Trial
F2	15FEB-25	Mohan Rao	Murder	High	Lakshmi Devi	Inspector	Bhimavaram PS	Charge Sheeted
F3	06-MAR-25	Priya Krishnan	Cybercrime	High	Lakshmi Devi	Inspector	Bhimavaram PS	Open

Q6. Criminals involved in FIR No. 2

```

SELECT cr.criminal_id, cr.name, cr.status
FROM CRIMINAL cr
JOIN FIR_CRIMINAL fc ON cr.criminal_id = fc.criminal_id
WHERE fc.fir_id='F2';

```

CRIMINAL_ID	NAME	STATUS
CR2	Sunil Das	Arrested

Q7. High severity crimes with FIR details

```

SELECT
c.crime_type, c.crime_date, c.location,
f.fir_id, f.case_status
FROM CRIME c
JOIN FIR f ON c.crime_id = f.crime_id
WHERE c.severity = 'High';

```

CRIME_TYPE	CRIME_DATE	LOCATION	FIR_ID	CASE_STATUS
------------	------------	----------	--------	-------------

Murder	14-FEB-25	Railway Station, Bhimavaram	F2	Charge Sheeted
Cybercrime	05-MAR-25	Online	F3	Open

Aggregate Queries

Q8. Count of cases by crime type

```
SELECT crime_type, COUNT(*) AS total_cases
FROM CRIME
GROUP BY crime_type
ORDER BY total_cases DESC;
```

CRIME_TYPE	TOTAL_CASES
Theft	1
Assault	1
Cybercrime	1
Murder	1

Q9. Officers handling more than 1 FIR

```
SELECT o.name, o.rank, COUNT(f.fir_id) AS fir_count
FROM OFFICER o
JOIN FIR f ON o.officer_id = f.officer_id
GROUP BY o.officer_id, o.name, o.rank
HAVING COUNT(f.fir_id) > 1;
```

NAME	RANK	FIR_COUNT
Lakshmi Devi	Inspector	2

Q10. Monthly crime report for 2025

```
SELECT EXTRACT(MONTH FROM crime_date) AS month,
       TO_CHAR(crime_date, 'Month') AS month_name,
       COUNT(*) AS total_crimes
FROM CRIME
WHERE EXTRACT(YEAR FROM crime_date) = 2025
GROUP BY EXTRACT(MONTH FROM crime_date), TO_CHAR(crime_date, 'Month')
ORDER BY month;
```

MONTH	MONTH_NAME	TOTAL_CRIMES
1	January	1
2	February	1
3	March	1
4	April	1

Sub-query and View

Q11. Criminals with prior records above average

```
SELECT name, prior_records
FROM CRIMINAL
WHERE prior_records > (SELECT AVG(prior_records) FROM CRIMINAL);
```

NAME	PRIOR_RECORDS
Ramesh Goud	2

Q12. Create a View for Active FIRs

```
CREATE VIEW Active_FIRs AS
SELECT f.fir_id, f.fir_date, c.crime_type,
o.name AS officer, f.case_status
FROM FIR f
JOIN CRIME c ON f.crime_id = c.crime_id
JOIN OFFICER o ON f.officer_id = o.officer_id
WHERE f.case_status NOT IN ('Closed');
SELECT * FROM Active_FIRs; ERROR
at line 1:
```

ORA-00942: table or view does not exist

FIR_ID	FIR_DATE	CRIME_TYPE	OFFICER	CASE_STATUS
F1	11-JAN-25	Theft	Ravi Kumar	Trial
F2	15-FEB-25	Murder	Lakshmi Devi	Charge Sheeted
F3	06-MAR-25	Cybercrime	Lakshmi Devi	Open

10.TESTING & PROJECT SUMMARY

10.1 TEST CASES

TC#	Test Scenario	SQL / Action	Expected Output	Status
TC-01	Insert a new FIR	INSERT INTO FIR ...	FIR record added	PASS
TC-02	Fetch Open cases	SELECT...WHERE status='Open'	Open FIR list returned	PASS
TC-03	Update FIR status	UPDATE FIR SET case_status=...	Status updated	PASS
TC-04	FK violation test	DELETE FIR without clearing junctions	FK Constraint Error raised	PASS
TC-05	Multi-table JOIN	JOIN FIR+CRIME+OFFICER+STATION	Combined result set	PASS
TC-06	Aggregate COUNT	GROUP BY crime_type	Crime count per type	PASS
TC-07	Sub-query	WHERE prior_records>AVG(...)	Filtered criminal list	PASS
TC-08	Create View	CREATE VIEW Active_FIRs ...	View created, queryable	PASS

10.2 SUMMARY REPORT

Project Summary

The Crime Record Management System successfully demonstrates the application of core DBMS concepts to a socially relevant domain. The project covered end-to-end database design

from ER modeling to complex SQL querying. All tables are normalized to 3NF, ensuring minimal redundancy and high data integrity.

Component	Details
Total Tables Created	8 (including 2 junction tables)
Sample Records Inserted	20+ across all tables
DDL Statements	CREATE TABLE, ALTER TABLE
DML Statements	INSERT, UPDATE, DELETE
DQL Queries Written	12 (Basic, JOIN, Aggregate, Sub-query, View)
Test Cases Executed	8 -- All Passed
Normalization Level	Third Normal Form (3NF)

10.3 LEARNING OUTCOMES

- Understood ER modeling and translation into relational tables.
- Gained hands-on experience writing DDL, DML, and DQL in SQL.
- Applied normalization to eliminate redundancy and improve integrity.
- Implemented referential integrity using Primary Keys and Foreign Keys.
- Wrote complex JOIN, GROUP BY, HAVING, Sub-query, and View statements.
- Designed a complete database for a real-world law enforcement scenario.

10.4 FUTURE ENHANCEMENTS

- Develop a web-based front-end using HTML/CSS/JavaScript with a Python/Java backend.
- Add biometric and fingerprint-based criminal identification support.
- Integrate with NCRB (National Crime Records Bureau) APIs.
- Implement role-based access control (RBAC) for secure officer login.
- Enable geo-tagging of crime locations for map-based visualization.
- Add automated deadline alerts for officers on pending investigations.

11.REFERENCES

1. Silberschatz, A Korth , H. F., &Sudarshan, S. (2020). Database System Concepts (7th edition.). McGraw-Hill Education.
2. W3Schools SQL Tutorial -- <https://www.w3schools.com/sql/>
3. National Crime Records Bureau (NCRB), India -- <https://ncrb.gov.in/>
4. Geeks for Geeks DBMS Articles -- <https://www.geeksforgeeks.org/dbms/>
5. Tutorials Point SQL Reference -- <https://www.tutorialspoint.com/sql/>